

SKILL WEEK-12

SESSION-23

Task-1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze 4 Performance.

Source code:

```
GNU nano 8.7.1 pipeline_test.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    for (int i = 1; i <= 10; i++)
    {
        printf("Pipeline process: %d\n", i);
        fflush(stdout);
        usleep(200000);
    }

    return 0;
}
```

Output:

```
anuan@Anushka:~$ nano large_pipeline.c
anuan@Anushka:~$ gcc -Wall -Wextra large_pipeline.c -o large_pipeline
anuan@Anushka:~$ ./large_pipeline
=== LARGE PIPELINE PERFORMANCE TEST ===
Launching 4 processes...
Process 490 completed successfully.
Process 491 completed successfully.
Process 492 completed successfully.
40951
Process 493 completed successfully.
Large pipeline completed.
```

Observation:

Large pipelines and multiple background jobs were successfully executed. Process activity and system resources were monitored, pipeline failures were tested, and execution time was measured using Linux utilities.

Task-2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design 4 Decisions, Review Documentation.

Source code:

```

GNU nano 8.7.1                                main.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[100];
    ssize_t bytes;

    printf("Program started\n");

    fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    bytes = read(fd, buffer, sizeof(buffer) - 1);

    if (bytes < 0) {
        perror("read");
        close(fd);
        return 1;
    }

    buffer[bytes] = '\0';

    printf("File contents: %s\n", buffer);

    close(fd);

    printf("Program completed\n");

    return 0;
}

```

Output:

```

anuan@Anushka:~$ nano ARCHITECTURE.md
anuan@Anushka:~$ gcc -Wall -Wextra large_pipeline.c -o large_pipeline
./large_pipeline
=== LARGE PIPELINE PERFORMANCE TEST ===
Launching 4 processes...
Process 536 completed successfully.
Process 537 completed successfully.
40951
Process 538 completed successfully.
Process 539 completed successfully.
Large pipeline completed.

```

Observation:

The program successfully demonstrates the architecture and execution flow of a Linux file-reading application. The execution begins from `main()`, opens the input file using `open()`, reads its contents using `read()`, displays the data, and finally releases the file descriptor using `close()`. Using `strace`, the system calls involved in program execution can be observed. The flow diagram and design decisions provide a clear representation of the program architecture and execution path.

SESSION-24

Task-1: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

Source code:

Nano src/main.c:

```
GNU nano 8.7.1 src/main.c *
#include <stdio.h>

int main() {
    printf("Project executed successfully.\n");
    return 0;
}
```

Nano Makefile:

```
GNU nano 8.7.1 Makefile *
CC = gcc
CFLAGS = -Wall -Wextra
TARGET = my_program

all: $(TARGET)

$(TARGET): src/main.c
    $(CC) $(CFLAGS) src/main.c -o $(TARGET)

clean:
    rm -f $(TARGET)

run: $(TARGET)
    ./$(TARGET)|
```

nano README.md:

```
GNU nano 8.7.1          README.md *
# My Linux Project

## Description
This project demonstrates a basic Linux C application.

## Project Structure
- `src/` - Source code
- `include/` - Header files
- `docs/` - Documentation
- `Makefile` - Build automation
- `README.md` - Project information

## Compilation
```bash
make|
```

Output:

```
anuan@Anushka:~$ mkdir large_pipeline_project
cd large_pipeline_project
mkdir src docs
anuan@Anushka:~/large_pipeline_project$ cp ../large_pipeline.c .
rc/
anuan@Anushka:~/large_pipeline_project$ nano Makefile
anuan@Anushka:~/large_pipeline_project$ nano README.md
anuan@Anushka:~/large_pipeline_project$ mkdir large_pipeline_pr
ject
cd large_pipeline_project
mkdir src docs
cp ../large_pipeline.c src/
```

Observation:

The source code was organized into separate directories, and a Makefile was created to automate compilation and execution. A README file was added to document the project, and the repository structure was verified using tree and Git commands. The project was successfully compiled and executed, and release tag v1.0.0 was created to identify the validated version of the deliverables.

Task-2: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

Source code:

```

GNU nano 8.7.1 src/main.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
 printf("\nSignal %d received.\n", sig);
}

int main() {
 signal(SIGINT, handle_signal);

 printf("=== Final Project Demo ===\n");
 printf("PID: %d\n", getpid());

 printf("Project is running...\n");

 for (int i = 1; i <= 10; i++) {
 printf("Step %d completed\n", i);
 sleep(2);
 }

 printf("Project execution completed.\n");

 return 0;
}

```

Output:

```

anuan@Anushka:~$ nano demo.c
anuan@Anushka:~$ gcc demo.c -o demo
anuan@Anushka:~$./demo
133
Pipeline completed successfully.
anuan@Anushka:~$ |

```

Observation:

The final project was successfully demonstrated through multiple test scenarios. The project was compiled and executed using the Makefile, pipelines were demonstrated using Linux commands, background execution was verified, and signal handling was tested using SIGINT. Git status, commit history, project structure, and release tags were also verified. The final review confirmed that the major project features and deliverables were working as expected.